

Lab 2.1: Implementation of DFA to identify strings having a's and b's in multiples of 3 in sequence

Background and Requirements

In **Formal Language Theory** and **Automata Theory**, a Deterministic Finite Automaton (DFA) is a mathematical model used to recognize whether a string belongs to a particular language. A DFA processes an input string one character at a time and moves through a finite number of states according to predefined transition rules. After reading the entire input, the string is accepted if the automaton ends in an accepting state; otherwise, it is rejected.

DFAs are widely used in compiler design, lexical analysis, pattern recognition, text processing, and digital system design because they provide an efficient method for validating strings against specific rules.

The task is to accept a string consisting of the symbols '**a**' and '**b**' and determine whether the string has a's and b's in multiples of 3 in sequence.

Source Code

```
alphabet = {"a", "b"}
s = input("Enter a string sequence: ")
state = "A"

for ch in s:
    if ch not in alphabet:
        state = "dead"
        break
    match state:
        case "A":
            state = "B" if ch == "a" else "E"
        case "B":
            if ch == "a":
                state = "C"
            else:
                state = "dead"
                break
        case "C":
            if ch == "a":
                state = "D"
            else:
                state = "dead"
                break
        case "D":
            state = "B" if ch == "a" else "E"
        case "E":
            if ch == "b":
                state = "F"
            else:
                state = "dead"
                break
        case "F":
```

```
if ch == "b":  
    state = "D"  
else:  
    state = "dead"  
    break
```

```
if state == "D":  
    print("Valid")  
else:  
    print("Invalid")
```

Output/Observations

```
Enter a string sequence: aaabbbaaa  
Valid  
○ (base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\TOC>
```

```
Enter a string sequence: ababaaa  
Invalid  
❖ (base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works\TOC>
```

Lab 2.2: Implementation of DFA to identify strings having 0's or 1's in multiples of 3

Source Code

```
alphabet = {"0", "1"}
s = input("Enter a string sequence: ")

state = "A"

for ch in s:
    if ch not in alphabet:
        state = "Invalid"
        break

    match state:
        case "A":
            state = "B" if ch == "0" else "D"

        case "B":
            state = "C" if ch == "0" else "E"

        case "C":
            state = "A" if ch == "0" else "F"

        case "D":
            state = "E" if ch == "0" else "G"

        case "E":
            state = "F" if ch == "0" else "H"

        case "F":
            state = "D" if ch == "0" else "I"
```

```
case "G":
```

```
    state = "H" if ch == "0" else "A"
```

```
case "H":
```

```
    state = "I" if ch == "0" else "B"
```

```
case "I":
```

```
    state = "G" if ch == "0" else "C"
```

```
if state in ('ABCDG'):
```

```
    print("Valid")
```

```
else:
```

```
    print("Invalid")
```

Output/Observations

```
Enter a string sequence: 01010
Valid
○ (base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works>
```

```
Enter a string sequence: 010
Invalid
○ (base) PS D:\Prasoon Raj Shakya\BSc. CSIT\4th sem\Lab works>
```

Conclusion

This lab demonstrated the practical implementation of Deterministic Finite Automata (DFA) to validate strings based on specific language constraints. By designing state transitions and testing different input strings, the experiments reinforced the concepts of finite-state machines, transition functions, and acceptance criteria, highlighting how DFAs efficiently recognize patterns defined by formal languages.